

Elementary Image Processing with Shaders

Image Processing

- Our first look at *GPGPU*
 - General-*P*urpose computation on *G*raphics *P*rocessing *U*nits
- Input: Image
- Output: Processed image
- A *kernel* runs on each pixel

Image Processing Examples

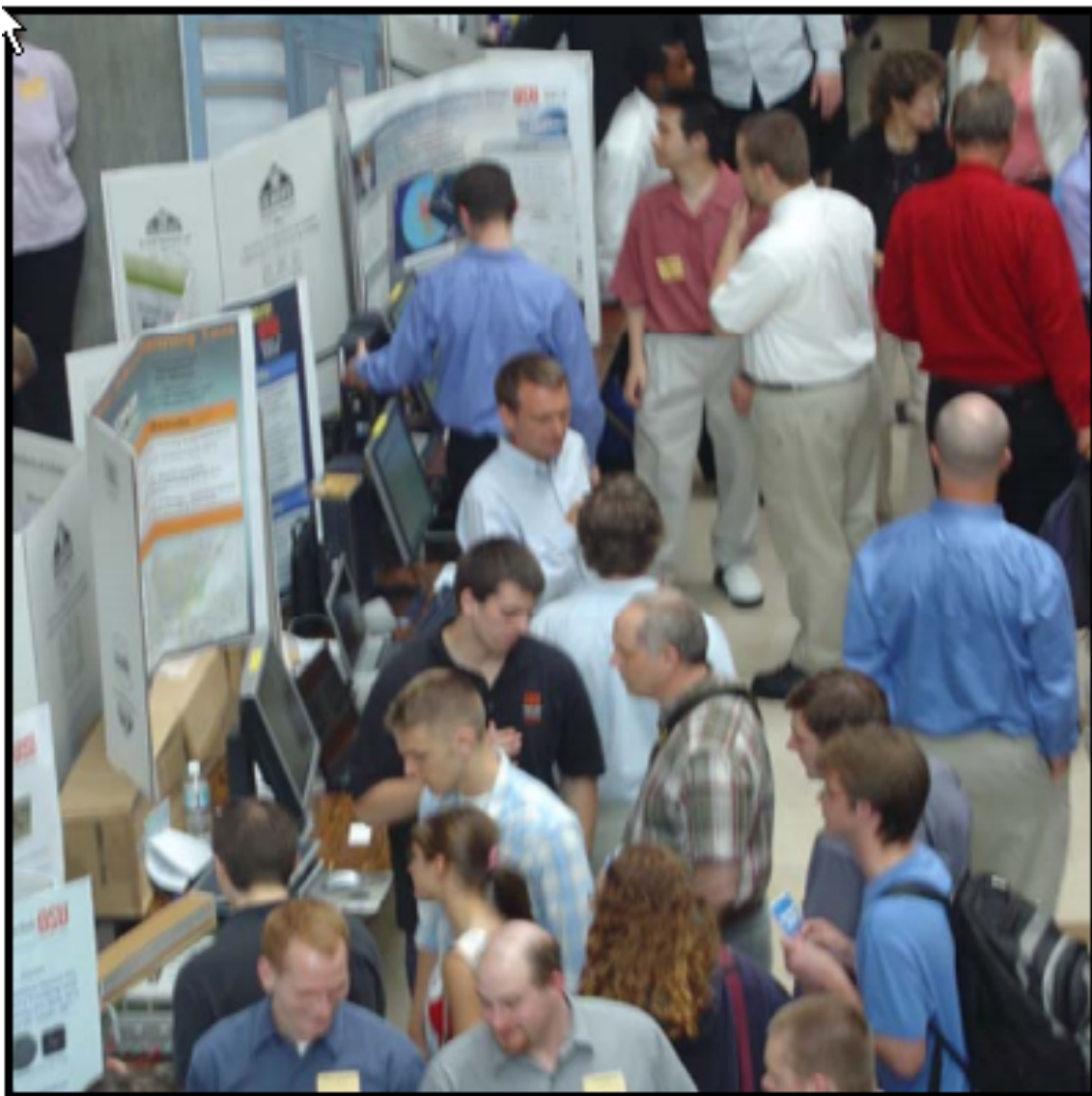
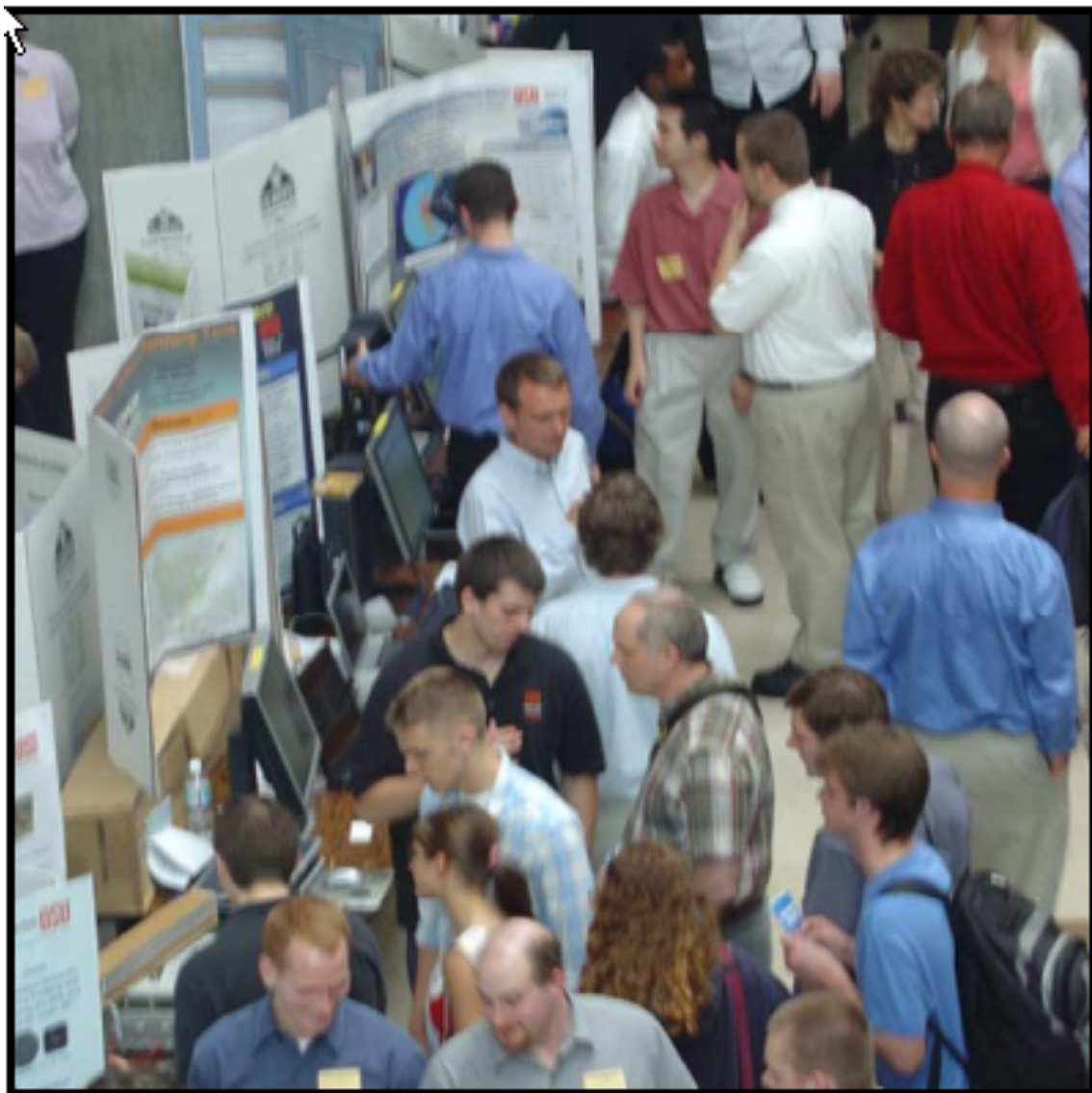


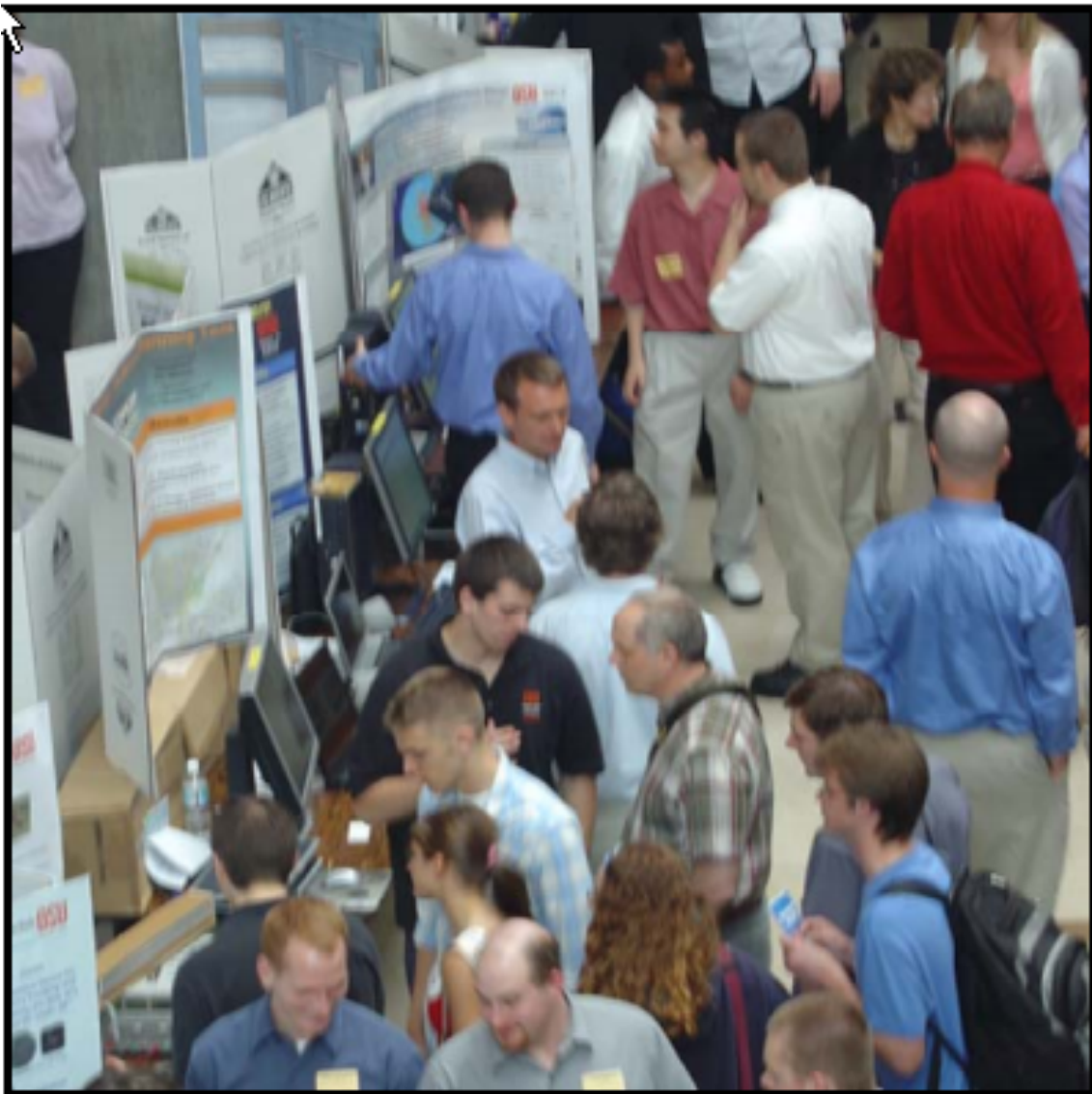
Image Negative

Image Processing Examples



Edge Detection

Image Processing Examples



Toon Rendering



Image Processing Questions

- Is the GPU a good fit for image processing?
- Is image processing *data-parallel*?
- What about *bus traffic*?
- What type of shader should implement an image processing kernel?

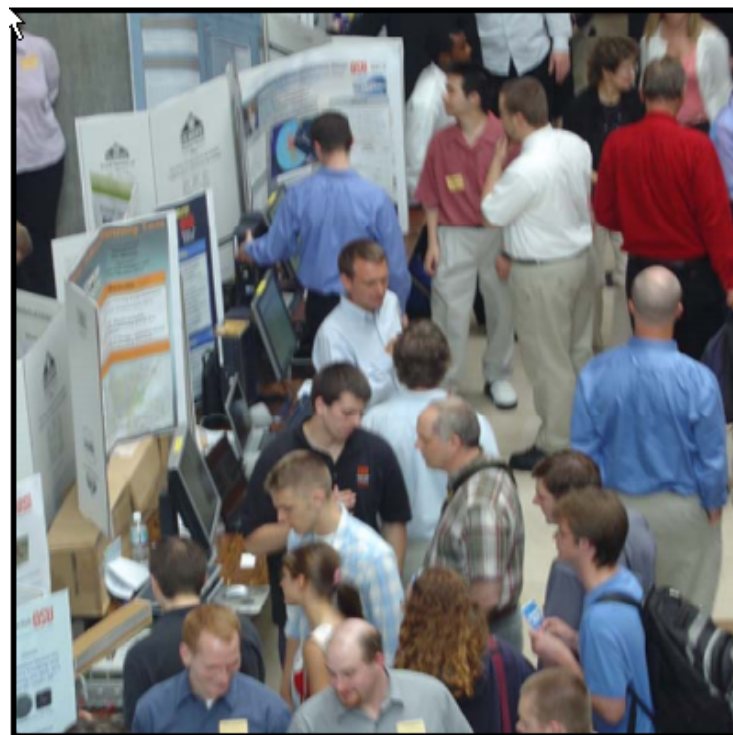
Image Processing: GPU Setup

- Input:
 - Texture
 - *Viewport-aligned quad*, a.k.a *full-screen quad*
- Output: framebuffer
 - ...for now
- Kernel: fragment shader

Image Processing: GPU Setup

3. Each fragment shader can access any part of the image stored as a texel:

gather



4. Each fragment shader executes the kernel and writes the color to the framebuffer



Fragment
shader

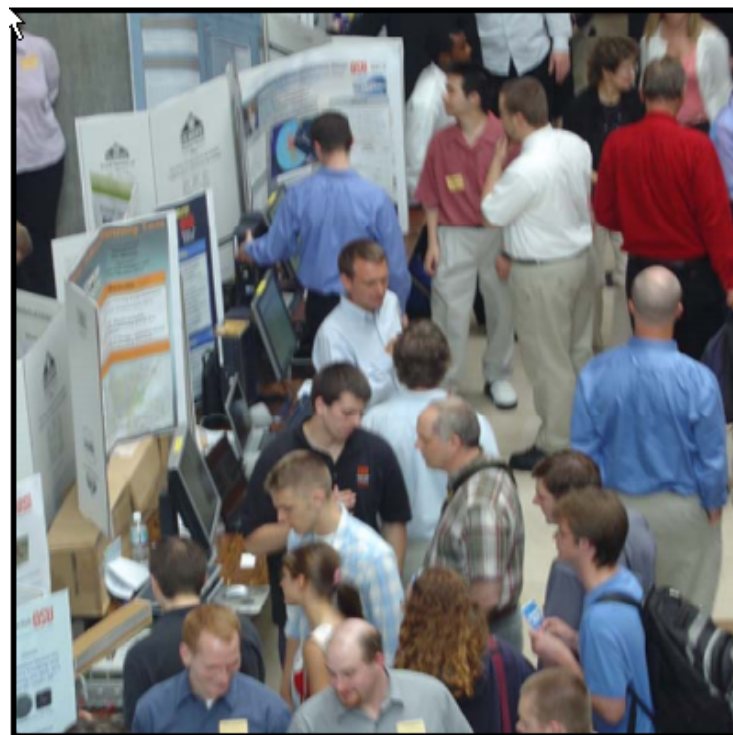
2. A fragment is invoked
for each screen pixel

1. Render viewport-aligned quad

Image Processing: GPU Setup

3. Each fragment shader can access any part of the image stored as a texel:

gather



4. Each fragment shader executes the kernel and writes the color to the framebuffer



Fragment
shader

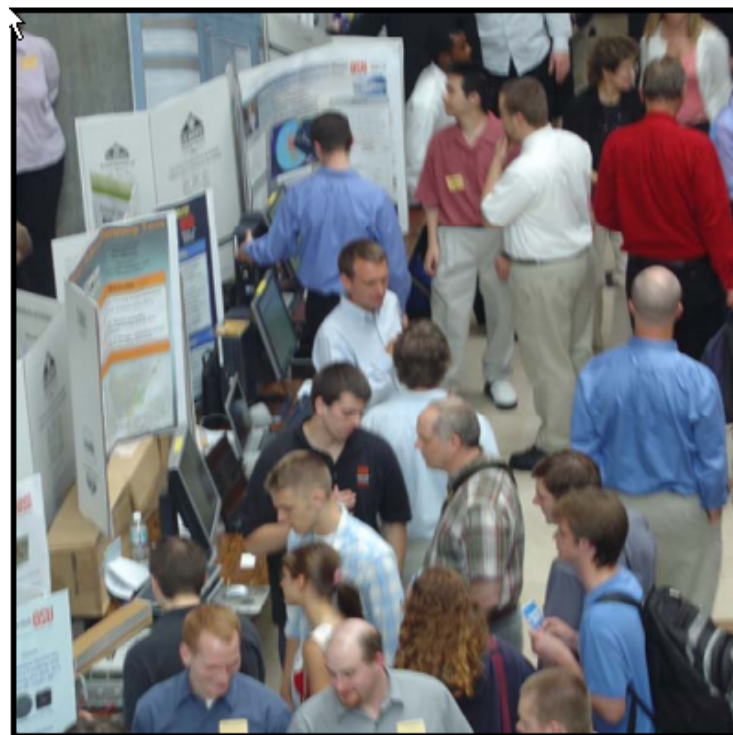
2. A fragment is invoked
for each screen pixel

1. Render viewport-aligned quad

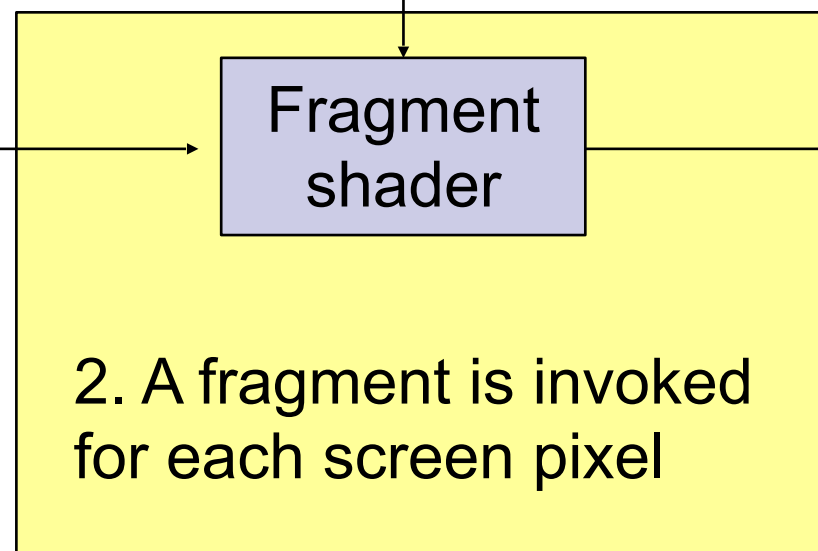
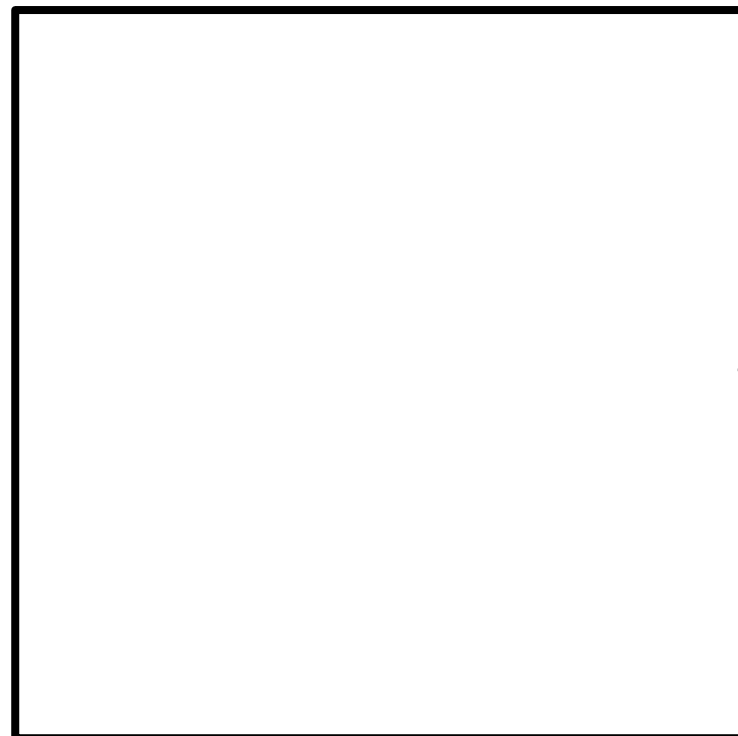
Image Processing: GPU Setup

3. Each fragment shader can access any part of the image stored as a texel:

gather



4. Each fragment shader executes the kernel and writes the color to the framebuffer

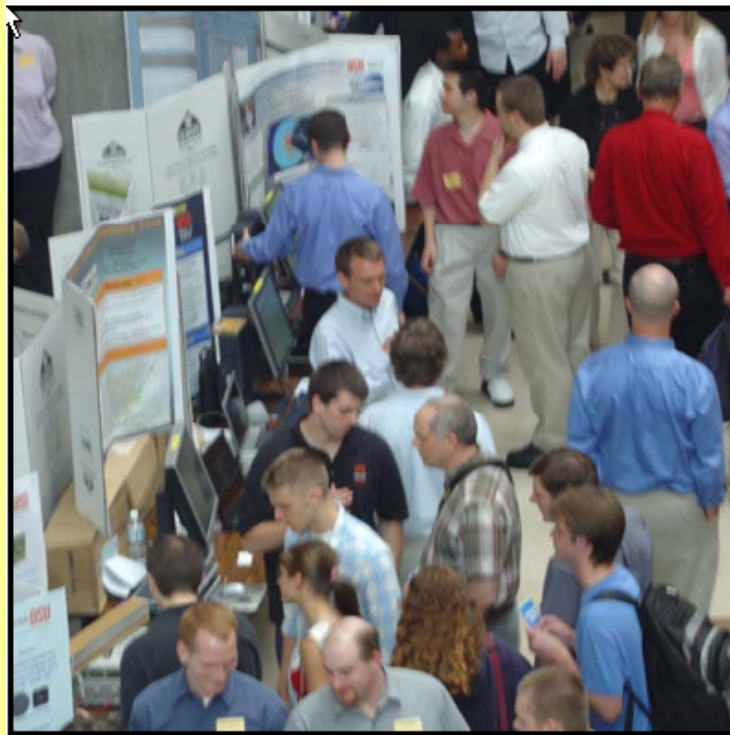


1. Render viewport-aligned quad

Image Processing: GPU Setup

3. Each fragment shader can access any part of the image stored as a texel:

gather



4. Each fragment shader executes the kernel and writes the color to the framebuffer



Fragment
shader

2. A fragment is invoked
for each screen pixel

1. Render viewport-aligned quad

Image Processing: GPU Setup

3. Each fragment shader can access any part of the image stored as a texel:

gather



4. Each fragment shader executes the kernel and writes the color to the framebuffer



Fragment
shader

2. A fragment is invoked
for each screen pixel

1. Render viewport-aligned quad

Image Processing: GPU Setup

- Store texture coordinates per vertex

Vertex Shader

```
in vec3 Position;  
in vec2 Texcoords;  
out vec2 fs_Texcoords;  
  
void main(void)  
{  
    fs_Texcoords = Texcoords;  
    gl_Position = vec4(Position, 1.0);  
}
```

Fragment Shader

```
uniform sampler2D u_Image;  
  
in vec2 fs_Texcoords;  
out vec4 out_Color;  
  
void main(void)  
{  
    out_Color = texture(u_Image,  
        fs_Texcoords);  
}
```

Image Processing: GPU Setup

- Compute texture coordinate in fragment shader

Vertex Shader

```
in vec3 Position;  
void main(void)  
{  
    gl_Position = vec4(Position, 1.0);  
}
```

Fragment Shader

```
uniform sampler2D u_Image;  
uniform vec2 u_inverseViewportDimensions;  
  
out vec4 out_Color;  
  
void main(void)  
{  
    vec2 txCoord = u_inverseViewportDimensions *  
        gl_FragCoord.xy;  
  
    out_Color = texture(u_Image, txCoord);  
}
```


Image Processing: GPU Setup

- Compute texture coordinate in fragment shader

Vertex Shader

```
in vec3 Position;  
void main(void)  
{  
    gl_Position = vec4(Position, 1.0);  
}
```

What is `u_inverseViewportDimensions`?

Fragment Shader

```
uniform sampler2D u_Image;  
uniform vec2 u_inverseViewportDimensions;  
  
out vec4 out_Color;  
  
void main(void)  
{  
    vec2 txCoord = u_inverseViewportDimensions *  
        gl_FragCoord.xy;  
  
    out_Color = texture(u_Image, txCoord);  
}
```

Image Processing: GPU Setup

- How do you access adjacent texels?

```
uniform sampler2D u_Image;
```

```
in vec2 fs_Texcoords;  
out vec4 out_Color;
```

```
void main(void)  
{  
    vec4 c0 = texture(u_Image, fs_Texcoords);  
    vec4 c1 = textureOffset(u_Image, fs_Texcoords, ivec2(-1, 0));  
    vec4 c2 = textureOffset(u_Image, fs_Texcoords, ivec2( 1, 0));  
    vec4 c3 = textureOffset(u_Image, fs_Texcoords, ivec2( 0, -1));  
    vec4 c4 = textureOffset(u_Image, fs_Texcoords, ivec2( 0, 1));  
  
    out_Color = (c0 + c1 + c2 + c3 + c4) * 0.2;  
}
```

Image Processing: GPU Setup

- How do you access adjacent texels?

```
uniform sampler2D u_Image;
```

```
in vec2 fs_Texcoords;  
out vec4 out_Color;
```

```
void main(void)
```

```
{  
    vec4 c0 = texture(u_Image, fs_Texcoords);  
    vec4 c1 = textureOffset(u_Image, fs_Texcoords, ivec2(-1, 0));  
    vec4 c2 = textureOffset(u_Image, fs_Texcoords, ivec2( 1, 0));  
    vec4 c3 = textureOffset(u_Image, fs_Texcoords, ivec2( 0, -1));  
    vec4 c4 = textureOffset(u_Image, fs_Texcoords, ivec2( 0, 1));  
  
    out_Color = (c0 + c1 + c2 + c3 + c4) * 0.2;  
}
```

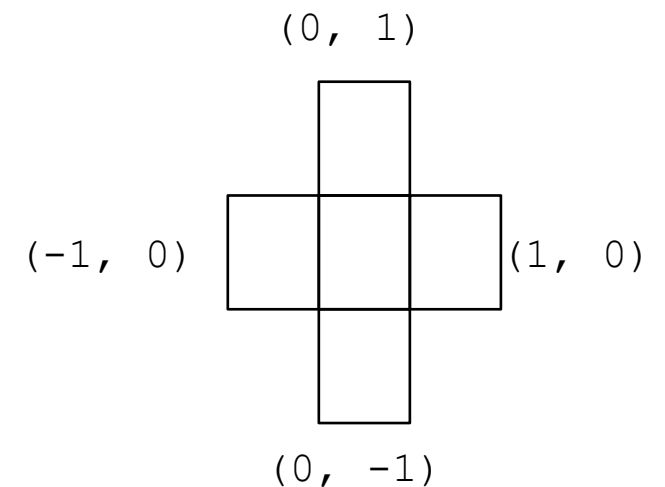


Image Processing: GPU Setup

- `textureOffset` requires constants
 - E.g. `ivec2(x, y)` is not allowed
- How else do you access adjacent texels?

Image Processing: GPU Setup

- How else do you access adjacent texels?

```
uniform sampler2D u_Image;
uniform vec2 u_inverseViewportDimensions;

out vec4 out_Color;

void main(void)
{
    vec2 txCoord = u_inverseViewportDimensions * gl_FragCoord.xy;
    vec2 delta = 1.0 / textureSize(u_Image);

    vec4 c0 = texture(u_Image, txCoord);
    vec4 c1 = texture(u_Image, txCoord + (delta * vec2(-1.0, 0.0)));
    vec4 c2 = texture(u_Image, txCoord + (delta * vec2( 1.0, 0.0)));
    vec4 c3 = texture(u_Image, txCoord + (delta * vec2( 0.0, -1.0)));
    vec4 c4 = texture(u_Image, txCoord + (delta * vec2( 0.0, 1.0)));

    out_Color = (c0 + c1 + c2 + c3 + c4) * 0.2;
}
```

Image Processing: GPU Setup

- How else do you access adjacent texels?

```
uniform sampler2D u_Image;
uniform vec2 u_inverseViewportDimensions;

out vec4 out_Color;

void main(void)
{
    vec2 txCoord = u_inverseViewportDimensions * gl_FragCoord.xy;
    vec2 delta = 1.0 / textureSize(u_Image);

    vec4 c0 = texture(u_Image, txCoord);
    vec4 c1 = texture(u_Image, txCoord + (delta * vec2(-1.0, 0.0)));
    vec4 c2 = texture(u_Image, txCoord + (delta * vec2(1.0, 0.0)));
    vec4 c3 = texture(u_Image, txCoord + (delta * vec2(0.0, -1.0)));
    vec4 c4 = texture(u_Image, txCoord + (delta * vec2(0.0, 1.0)));

    out_Color = (c0 + c1 + c2 + c3 + c4) * 0.2;
}
```


Image Processing: GPU Setup

■ How else do you access adjacent texels?

```
uniform sampler2D u_Image;  
uniform vec2 u_inverseViewportDimensions;
```

```
out vec4 out_Color;
```

```
void main(void)
```

```
{
```

```
    vec2 txCoord = u_inverseViewportDimensions * gl_FragCoord.xy;
```

```
    vec2 delta = 1.0 / textureSize(u_Image);
```

```
    vec4 c0 = texture(u_Image, txCoord);
```

```
    vec4 c1 = texture(u_Image, txCoord + (delta * vec2(-1.0, 0.0)));
```

```
    vec4 c2 = texture(u_Image, txCoord + (delta * vec2( 1.0, 0.0)));
```

```
    vec4 c3 = texture(u_Image, txCoord + (delta * vec2( 0.0, -1.0)));
```

```
    vec4 c4 = texture(u_Image, txCoord + (delta * vec2( 0.0, 1.0)));
```

```
    out_Color = (c0 + c1 + c2 + c3 + c4) * 0.2;
```

```
}
```

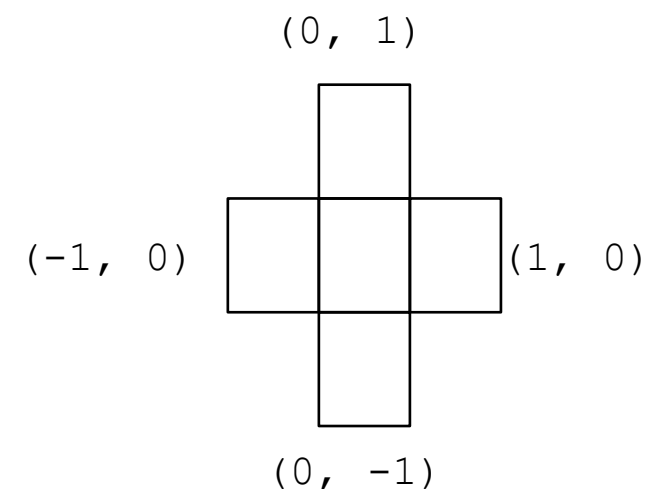


Image Processing: Kernel Examples

■ Image Negative

```
uniform sampler2D u_Image;
```

```
in vec2 fs_Texcoords;
```

```
out vec4 out_Color;
```

```
void main(void)
```

```
{
```

```
    out_Color = vec4(1.0) - texture(u_Image, fs_Texcoords);
```

```
}
```

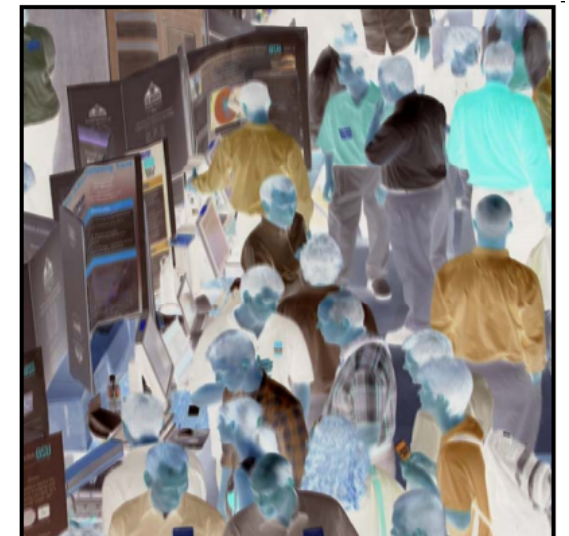


Image Processing: Kernel Examples

- Gaussian Blur

2D Gaussian

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

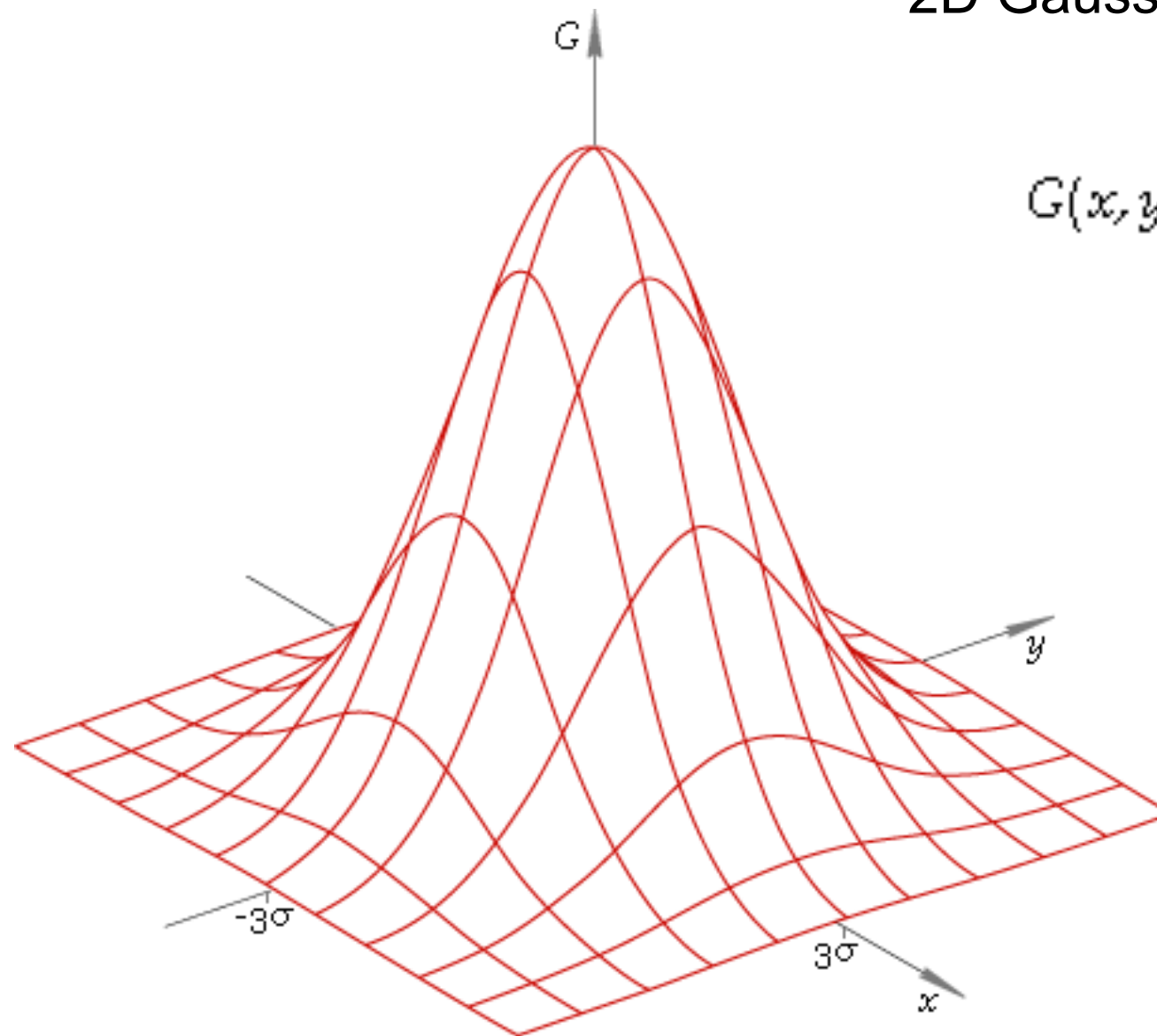


Image Processing: Kernel Examples

- Filter for 3x3 Gaussian Blur:

$$\frac{1}{16} * \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

- The elements add to one
- Other filters are also used for
 - Edge detection
 - Sharpen
 - Emboss
 - ...



Image Processing: Kernel Examples

- Gaussian Blur
 - How would you implement the fragment shader?
 - How is the memory coherence?
 - 3x3, 5x5, etc.

Image Processing: Kernel Examples

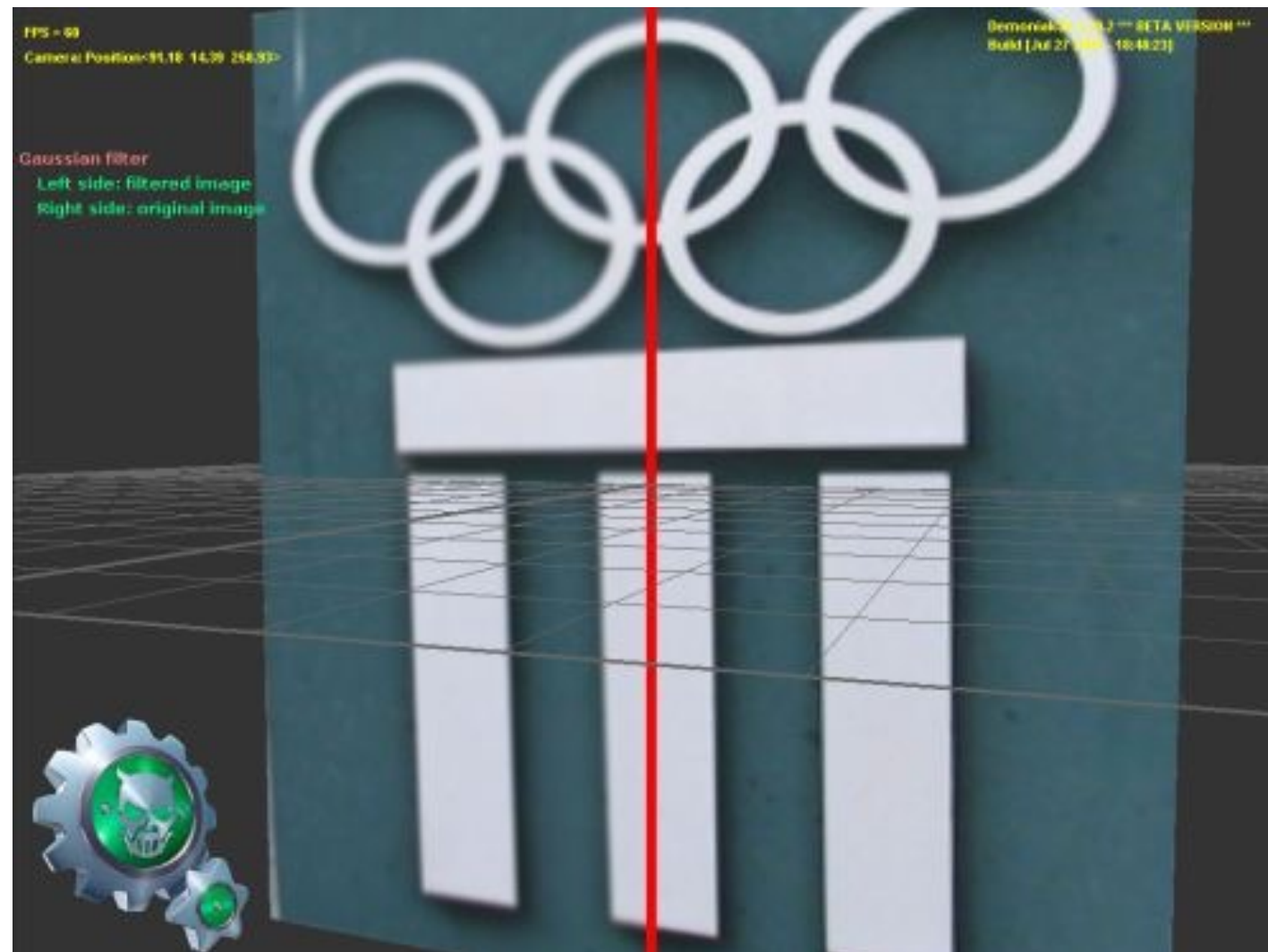


Image Processing: Kernel Examples

- What does this filter do?

$$\frac{1}{9} * \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Image Processing: Read backs

- `glReadPixels`

```
glUseProgram(/* ... */);  
glDraw* (/* ... */);
```

```
unsigned char rgb = new unsigned char[width * height * 3];
```

```
glReadPixels(0, 0, width, height, GL_RGB, GL_UNSIGNED_BYTE, rgb);
```

```
// ...  
delete [] rgb;
```


Image Processing: Read backs

- `glReadPixels`

```
glUseProgram(/* ... */);  
glDraw* (/* ... */);
```

Use GPU for image processing

```
unsigned char rgb = new unsigned char[width * height * 3];
```

```
glReadPixels(0, 0, width, height, GL_RGB, GL_UNSIGNED_BYTE, rgb);
```

```
// ...
```

```
delete [] rgb;
```

Image Processing: Read backs

■ glReadPixels

```
glUseProgram(/* ... */);  
glDraw* (/* ... */);
```

Allocate buffer for processed image

```
unsigned char rgb = new unsigned char[width * height * 3];
```

```
glReadPixels(0, 0, width, height, GL_RGB, GL_UNSIGNED_BYTE, rgb);
```

```
// ...  
delete [] rgb;
```

Image Processing: Read backs

■ glReadPixels

```
glUseProgram(/* ... */);  
glDraw* (/* ... */);
```

```
unsigned char rgb = new unsigned char[width * height * 3];
```

```
glReadPixels(0, 0, width, height, GL_RGB, GL_UNSIGNED_BYTE, rgb);
```

```
// ...  
delete [] rgb;
```

Ask for framebuffer's color buffer

Image Processing: Read backs

- `glReadPixels`

```
glUseProgram(/* ... */);  
glDraw* (/* ... */);
```

```
unsigned char rgb = new unsigned char[width * height * 3];
```

```
glReadPixels(0, 0, width, height, GL_RGB, GL_UNSIGNED_BYTE, rgb);
```

```
// ...
```

```
delete [] rgb;
```

← You guys are sharp

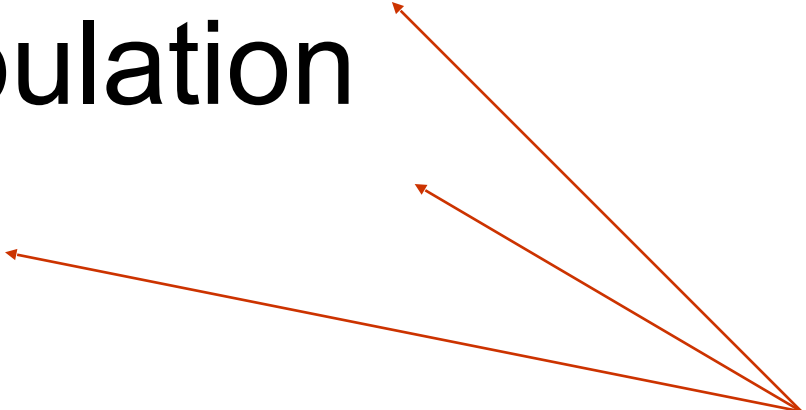


Image Processing: Read backs

- What is the major problem with `glReadPixels`?

Image Processing: Use Cases

- Photoshop-type applications
- Post-processing in games
- On the fly video manipulation
- Augmented reality



These last three don't even
need read backs

Homework

- Import any image
 - Output filtered image
- Be able to select filtering methods
- At least 3 filtering method mentioned in the class should be implemented
- Submit with short report

Summary

- Image Processing with Shader